

API REST para quem só sabe front-end

Se você só mexeu com HTML, CSS e JavaScript no navegador, tudo bem — este material assume que você **nunca** encostou em back-end. A ideia é entender o suficiente para conseguir *consumir* uma API REST em um projeto de front-end.

1. Por que o front-end precisa de uma API?

Seu site roda no navegador do usuário. Mas dados como "lista de produtos", "usuários cadastrados" ou "pedidos" ficam guardados em um banco de dados, em um servidor, em outro lugar do mundo.

O front-end não acessa esse banco de dados diretamente — isso seria um problema enorme de segurança (imagina expor a senha do banco no seu JavaScript!). Em vez disso, existe um **servidor de back-end** no meio do caminho, que:

1. Recebe pedidos do front-end
2. Busca ou grava dados no banco
3. Devolve uma resposta para o front-end

Esse "servidor no meio" geralmente expõe uma **API** — uma porta de entrada padronizada para pedir e enviar dados.

Front-end (navegador) <-- pedidos/respostas --> API (back-end) <-- consultas --> Banco de dados

2. O que significa "REST"?

REST é só um **estilo de organizar** essa API, usando o protocolo HTTP (o mesmo que seu navegador já usa para carregar páginas). Não é uma linguagem nem uma ferramenta — é uma convenção que a maioria das APIs modernas segue.

As ideias centrais:

- Tudo gira em torno de **recursos** (um recurso = uma "coisa": um usuário, um produto, um pedido).
- Cada recurso tem uma **URL** própria (chamada de *endpoint*).
- Você usa **verbos HTTP** diferentes para dizer o que quer fazer com aquele recurso (ler, criar, editar, apagar).
- O servidor responde com um **código de status** (dizendo se deu certo ou não) e geralmente com dados em **JSON**.

3. Os verbos HTTP (o "CRUD" da API)

Verbo HTTP	Para que serve	Equivalente no CRUD
GET	Buscar/ler dados	Read
POST	Criar um novo dado	Create
PUT / PATCH	Atualizar um dado existente	Update
DELETE	Apagar um dado	Delete

Exemplo com um recurso "produtos" (/produtos):

Ação	Método	URL
Listar todos os produtos	GET	/produtos
Ver um produto específico	GET	/produtos /12
Criar um produto novo	POST	/produtos
Atualizar o produto 12	PUT	/produtos /12
Apagar o produto 12	DELETE	/produtos /12

Repare: a URL quase não muda — o que muda é o **verbo**. É isso que torna a API previsível.

4. Códigos de status HTTP

O servidor sempre devolve um número junto com a resposta, indicando o que aconteceu:

Faixa	Significado	Exemplos comuns
2xx	Sucesso	200 OK, 201 Created, 204 No Content

4xx Erro do cliente (você pediu algo errado)

400 Bad Request, 401 Unauthorized, 404 Not Found

5xx Erro do servidor

500 Internal Server Error

Regra prática: se começa com **2**, deu certo. Se começa com **4**, o problema é no que você mandou. Se começa com **5**, o problema é no servidor (não é sua culpa, mas você precisa tratar mesmo assim).

5. JSON: o formato dos dados

Quase toda API REST troca dados em **JSON** (JavaScript Object Notation) — que parece muito com um objeto JavaScript:

```
{  
  "id": 12,  
  "nome": "Teclado mecânico",  
  "preco": 249.90,  
  "disponivel": true  
}
```

Isso é só texto — quando você recebe no JavaScript, precisa converter para um objeto de verdade (o **fetch** já faz isso pra você, como veremos abaixo).

6. Consumindo uma API REST no front-end

6.1 Buscando dados (**GET**)

```
async function buscarProdutos() {  
  try {  
    const resposta = await fetch("https://api.exemplo.com/produtos");  
  
    if (!resposta.ok) {  
      throw new Error(`Erro na requisição: ${resposta.status}`);  
    }  
  
    const produtos = await resposta.json();  
    console.log(produtos);  
    return produtos;  
  } catch (erro) {  
    console.error("Falha ao buscar produtos:", erro);  
  }  
}
```

```
}  
}
```

Pontos-chave:

- `fetch` retorna uma **Promise**, por isso o `await`.
- `resposta.ok` é `true` quando o status é `2xx`.
- `resposta.json()` também é assíncrono — converte o corpo da resposta (texto) em objeto/array JavaScript.

6.2 Enviando dados (POST)

```
async function criarProduto(novoProduto) {  
  try {  
    const resposta = await fetch("https://api.exemplo.com/produtos", {  
      method: "POST",  
      headers: {  
        "Content-Type": "application/json",  
      },  
      body: JSON.stringify(novoProduto),  
    });  
  
    if (!resposta.ok) {  
      throw new Error(`Erro ao criar produto: ${resposta.status}`);  
    }  
  
    const produtoCriado = await resposta.json();  
    return produtoCriado;  
  } catch (erro) {  
    console.error(erro);  
  }  
}
```

```
criarProduto({ nome: "Mouse sem fio", preco: 89.9, disponivel: true });
```

Repare nas diferenças em relação ao `GET`:

- `method: "POST"` — diz qual verbo usar.
- `headers` — avisa o servidor que o corpo enviado é JSON.
- `body: JSON.stringify(...)` — converte o objeto JS em texto JSON antes de enviar (o caminho inverso do `resposta.json()`).

6.3 Atualizando (PUT) e apagando (DELETE)

```
// Atualizar  
async function atualizarProduto(id, dadosAtualizados) {
```

```
const resposta = await fetch(`https://api.exemplo.com/produtos/${id}`, {
  method: "PUT",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(dadosAtualizados),
});
return resposta.json();
}
```

```
// Apagar
async function apagarProduto(id) {
  const resposta = await fetch(`https://api.exemplo.com/produtos/${id}`, {
    method: "DELETE",
  });
  return resposta.ok;
}
```

7. Autenticação (o básico que todo aluno vai encontrar)

Muitas APIs exigem que você prove "quem é você" em cada requisição. O jeito mais comum é enviar um **token** no cabeçalho (**header**) **Authorization**:

```
async function buscarPedidosDoUsuario(token) {
  const resposta = await fetch("https://api.exemplo.com/pedidos", {
    headers: {
      "Authorization": `Bearer ${token}`,
    },
  });
  return resposta.json();
}
```

Esse **token** geralmente é obtido depois de um login (a API devolve o token, e o front-end guarda para usar nas próximas requisições — normalmente em memória ou em **localStorage**, dependendo do caso).

8. Erros comuns de quem está começando

- **Esquecer o **Content-Type: application/json** no **POST/PUT**** → o servidor pode não entender o corpo enviado.
- **Esquecer o **JSON.stringify()**** → você acaba mandando `[object Object]` em vez de JSON de verdade.

- **Não tratar erros** → se a API cair ou devolver **404**, o app quebra silenciosamente se você não checar **resposta.ok**.
 - **Confundir **fetch** síncrono com assíncrono** → sempre use **await** (ou **.then()**), o **fetch** nunca devolve o dado na hora.
 - **CORS**: às vezes o navegador bloqueia a requisição por segurança (Cross-Origin Resource Sharing). Isso é configurado do lado do back-end — se der erro de CORS, o problema não está no seu código de front-end.
-

9. Resumindo em uma frase

Uma API REST é uma porta de entrada em URLs, onde cada recurso tem um endereço, e você usa verbos HTTP (**GET**, **POST**, **PUT**, **DELETE**) para ler, criar, atualizar ou apagar dados, trocando informação em formato JSON.